STUDENT

0003-GBH

TENTAMEN

230109_DV2606_2110_Salstentamen

Kurskod	--
Bedömningsform	--
Starttid	09.01.2023 14:00
Sluttid	09.01.2023 19:00
Bedömningsfrist	--
PDF skapad	21.05.2024 13:15
Skapad av	Zandra Johansson

1 Describe the main characteristics of *single instruction stream, multiple data stream* (SIMD) and *multiple instruction stream, multiple data stream* (MIMD) parallel computers, and also point out some fundamental differences between them.

Further, present some advantage and disadvantage for each type of computer.

Finally, give some example of applications (with motivation) where SIMD and MIMD computers would be preferable over the other, respectively.

SIMD: Single instruction multiple data, this is what data parallalism basically is. In the case of multiple processors or threads, each thread gets the same instruction and applies it on different parts of the same dataset. This is akin to how CUDA kernels work, where a kernel is launched on different parts of the same dataset.

MIMD: Multiple instruction multiple dataset, this is where processors or threads get different instructions and apply them on different parts of the same or different dataset. Hence the multiple **instruction** multiple **data**

As mentioned above, fundamentally, one executes a single instruction on multiple different data whilst the other can execute differing instructions on multiple data, not even necessary the same dataset

Advantages/ Disadvantages of SIMD:

+ Very simple to work with as you only need to create one single set of instructions and send them out on an arbitrary dataset, for this to work, each thread or processor needs to have some unique identifying property such as threadID.

- It utilizes only one instruction at a time, therefore decreasing parallalism compared to MIMD.
- Also, requires some independency for the operation to be done on the dataset, i.e squaring every number in an array does not need to know any numbers before hand, each thread can work on it's seperate array position

Advantages/ Disadvantages of MIMD:

+ Executes more instructions simultaneously on the same or different datasets -> increased parallalism

- Increased complexity for the programmer to code and efficiently take care of different threads at the same time with different instructions.

- Usually needs some sync mechanism if two different insturctions depend on each other, otherwise it is preferred that some independency exists between the insturctions.

Uses for SIMD:

CUDA kernles like matrix multiplication (It is more SPMD) but the basic utility still applies where one instruction is sent to all threads and they all preform the same operation on different parts of the dataset

Uses for MIMD:

Task parallalism is a uses of MIMD computers where different tasks can be done parallely at the same time, such as pipelines or node computer systems where different calucations are done on different nodes all at the same time.

Ord: 364

Besvarad.

2 Describe (define) what we mean with *speedup* and *efficiency*, in the context of a parallel computer system.

Speedup is the ratio between the time for the "best" sequential algorithm to the time of the parallel implementation of said algorithm. It is calculated by dividing the time for the sequential algorithm and the time of the parallel algorithm.

Efficiency is how much work the parallel processors are doing on a part that the sequential version has to also be doing. In other words, how much work is every processor doing based on the whole problem set. This can be calculated by dividing speedup/ amount of processors. If speedup is 32, and amount of processors is 32, then every processor is doing the maximal amount of work for that algorithm.

Ord: 111

Besvarad.

3 When we parallelize a sequential application, e.g., by using threads or message-passing, so it can can execute in parallel on *N* processors, we often find that the parallel program is not *N* times faster. Describe two important reasons for that.

First, the current implementation of the parallel algorithm might not be the best at utilizing the full potential of all the cores, this can also mean that the sequential version is just too good and in that case it requires a lot of complex programming to get working efficiently on multiple cores.

Secondly, the problem set might not be large enough to justify the amount of overhead required for the parallel algorithm. This comes in the form of creating threads, managing threads, killing threads and so on. To see a performance increase near *N*, the problem set needs to be of sufficient size to allow the processors to unleash their full potential.

Ord: 112

Besvarad.

4 Sometimes we find that a system or application has a *superlinear speedup*. What does that mean? Describe two situations when it may occur.

Superlinear speedup is when the linear speedup is very high, this can occur on two levels:

Hardware:
Sometimes, the problem set when divided by the processors fit perfectly in each processors cache, eliminating many cache misses and increasing cache hits. The processors will have access to their required data almost instantly therefore increasing the speedup by a very large margin.

Algorithmically:
Sometimes the problem at hand is much more effective when employed on multiple processors, for example searching a tree. When searching a tree sequentially, the processor has to traverese the entire tree, node by node until it hits the designated target. However, when done in parallel, each processor can get part of the tree and search it that way. This increases the odds of finding the target and therefore speedingup the process.

Superlinear speedup should not be expected because it happens on rare cases, like with the problemset fitting perfectly in the cache.

Ord: 154

Besvarad.

5 Shared-memory multiprocessors exist in two different variants: UMA (Uniform Memory Access) and NUMA (Non-uniform Memory Access) multiprocessors. Describe the primary difference between them, and also give an example when one is preferable over the other.

UMA: Expects every memory access to be of uniform time, meaning that P1 access time to global memory should be exactly the same as P2 access time to global memory. This is seen in symmetric processor systems where every processor is equidistant from the memory bus.

NUMA: Mostly seen in distributed memory systems, where memory access of processors are not of uniform time because of latency and other network issues.

As mentioned before, it depends on the system. When building a distributed computers system for more computational horse-power with multiple nodes communicating over a network, it is much better to go with NUMA as it is almost garuenteed that memory access will not have uniform time. if building a local phyiscal compact computer then UMA is to be recommended because it is much easier in that case to build a system with UMA times.

Ord: 144

Besvarad.

6 *Cache coherence* is an important issue to handle in a shared-memory multiprocessor. Describe, by giving an example, why ensuring cache coherence is important. (1p)

Two main approaches exist to maintain cache coherence: *write-invalidate* and *write-update*. Briefly describe each of them, e.g., by drawing a simple picture. (2p)

Describe some advantage and disadvantage with each of the approaches. (1p)

a) Cache coherence is the concept of caches on multiple processors having the same "up to date" data. Meaning, if one processor executes an operation on some data on it's cache, that modified data should be reflected on all other caches to create coherence. This is very important so that all processors yield the correct result when doing calculations. The job of overseeing what caches need updating and so on is done by what is called a snooper.

b) Write-update:
Write update, as the name sounds, updates the caches of other processors directly after modifying the data in the cache, all other processors get notified that the data is dirty and fetch the new data instead. this allows for faster data updates between the caches as there is no need to wait for modified data from a processor to come back because it happens directly after an operation.

Write invalidate:
Write invalidate is the process of modifying some data on the cache and then invalidating it on all other caches, this means that the other caches can't use that data for any operations. That data stays invalidated until the cache of the modified data decides to update the other caches or its cache block needs to be switched out of some other need. Once the new data exists on global memory, the other processors get updated with it.

c)
Write-update results in many global memory accesses and can cause slowdown if many processors try to update the global memory at the same time leading to congestion. This method however eliminates the need for complex implementations of cache coherency like in the case of write invalidate, which leads to protocols like MSI (modified, shared, invalid) which is a disadvantage of write invalidate. The advantage however is eliminating the need for so many global memory accesses reducing congestion and latency.

Ord: 308

Besvarad.

7 When writing a parallel application, we can use mainly two approaches: *task (a.k.a. functional) parallelism* and *data parallelism*.

- Describe the main characteristics of each of the two approaches.
- Further, describe some advantage and disadvantage with each of them.
- Finally, give some examples of applications (with motivation) that are suitable for task parallelism and data parallelism, respectively.

a) Data parallelism: same instruction done by different threads/ processors on the same dataset but on different parts of it, each thread/ processor takes care of a part.

Task parallelism: A task that is decomposed into sub-tasks where each thread/ processor usually takes care of one sub-task. This can mean that different threads are doing work on different datasets, or doing work in different functions.

b) Advantages/ Disadvantages Data parallelism:

+ Effective and simple since the same instruction is applied on a singular dataset but on different parts of it
+ Increased parallelism, operations done sequentially can be done much faster now instead.
- One instruction executed at a time, not as parallel as task parallelism for example.
- Data should not be dependent on each other, otherwise it is hard to execute the same instruction on the whole dataset in parallel

Advantages/ Disadvantages Task parallelism:

+ Increased parallelism over the whole program, not just a single instruction because now, tasks are decomposed into smaller tasks making the completion of one task go much faster.
- Increased complexity of managing threads with different jobs, harder to debug
- Some sort of sync mechanism needs to be implemented if tasks are dependant of each other

c) Data parallelism can be done on applications like gauss-elimination where each thread takes care of a row in the matrix to gauss eliminate for example.

Task parallelism can be done in pipelining for example where the whole pipeline is decomposed into smaller steps where each task can operate in parallel to each other. Some sync mechanism needs to implemented but once the pipeline is at full capacity, each subtask can operate on it's part of the data all at the same time.

Ord: 287

Besvarad.

8 Consider the sequential C code below for performing matrix-vector multiplication.

```
1  #define SIZE 4096
2  static double a[SIZE][SIZE], x[SIZE], y[SIZE];
3
4  static void init_values(void)
5  {
6      ... // init matrix a and vector x
7  }
8
9  static void matvec_seq(void)
10 {
11   int i, j;
12   for (i = 0; i < SIZE; i++) {
13     y[i] = 0.0;
14     for (j = 0; j < SIZE; j++)
15       y[i] = y[i] + a[i][j] * x[j];
16   }
17 }
18
19 int main(int argc, char **argv)
20 {
21   init_values();
22   matvec_seq();
23 }
```

Write (in C-like pseudo code) a parallel version of the matrix-vector multiplication algorithm, i.e., the function *matvec_seq()*, using a *1-dimensional partitioning*. Clearly state which assumptions you do about the type of machine and programming model. Assume that the number of processors is significantly fewer than SIZE. (2p)

Analyze your solution from a performance point of view, e.g., approximate speed-up, load-balancing, etc. (2p)

Let us assume we have 32 threads to work with, the processor has 32 cores and 32 gb of ram

Pseudo code:
SIZE = 4096
Define matrix: SIZE x SIZE
Define vector: SIZE
Define resultVector: SIZE
work = 0 //global
initiateMatrix() //global
initiateVector() //global
startWork(threadID): //Blockwise partioning
    amountOfWork = SIZE/THREADS
    start = ThreadID * amountOfWork
    for(int i = start; i < i + amountOfWork; i++)
      for(int j= 0; j < SIZE; j++)
        resultVector[i] += vector[j] * matrix[i * SIZE + j]
  work.atomicadd(1) // this will allow is exit the current thread and notify the main thread that it has finished //working
  pthread[threadID].exit()
main():

```
initiateMatrix()
initiateVector()
for loop:
  call 32 threads, with args: threadID
while work != 32
 //busy wait
printResults()
```

Since 4096 is equally divided by 32, we can assume that each thread will get the same amount of rows to work with, and because we exit the thread when it is done with it's work, we won't have overhead from "dead" threads, so from a load balancing perspective we have more or less equal balance.

The speedup might not be as large as we hope because we are busy waiting with the main thread until all threads are finished, and the parallel implementation is not as effective as one would hope. Therefore i believe we will get atleast a 24 speed up since we are utilizing 32 threads. And we know that it is very hard to achieve speedup = processors, because of ahmdal's law and the fact that implementation is not that great. But with 32 cores, 24x should be feasible

Ord: 263

Besvarad.

9 Thread Blocks and Warps

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- [ ] Threads in the same Warp can execute on different Streaming Multiprocessors.
- [ ] Threads from different Thread Blocks can execute in the same Warp.
- [x] A Thread Block can contain many Warps.
- [ ] A Thread Block cannot be larger than a Warp.
- [x] Threads in the same Warp cannot execute different instructions at the same time.

Rätt. 2 av 2 poäng.

10

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- [ ] Context switching on a GPU is normally faster than context switching on a CPU. ✔
- [x] The processor cores in a CPU normally executes in SIMD mode. ✖
- [ ] The processor cores on a CPU are normally organized in streaming multiprocessors.
- [ ] A CPU normally has more processor cores than a GPU
- [x] A CPU normally has larger caches than a GPU ✔

Delvis rätt. 0 av 2 poäng.

11

Select one or more alternatives. Selecting an alternative that is incorrect will give a negative score. The minimum score on this assignment is 0.

- [ ] All the threads in a thread block see the same registers
- [ ] The CPU can access variables in GPU shared memory
- [ ] Accesses to GPU global memory are faster than accesses to GPU shared memory.
- [x] The content of the CUDA global memory is not changed between two kernel launches in the same program ✔
- [x] Threads in different thread blocks cannot access the same variable in GPU shared memory ✔

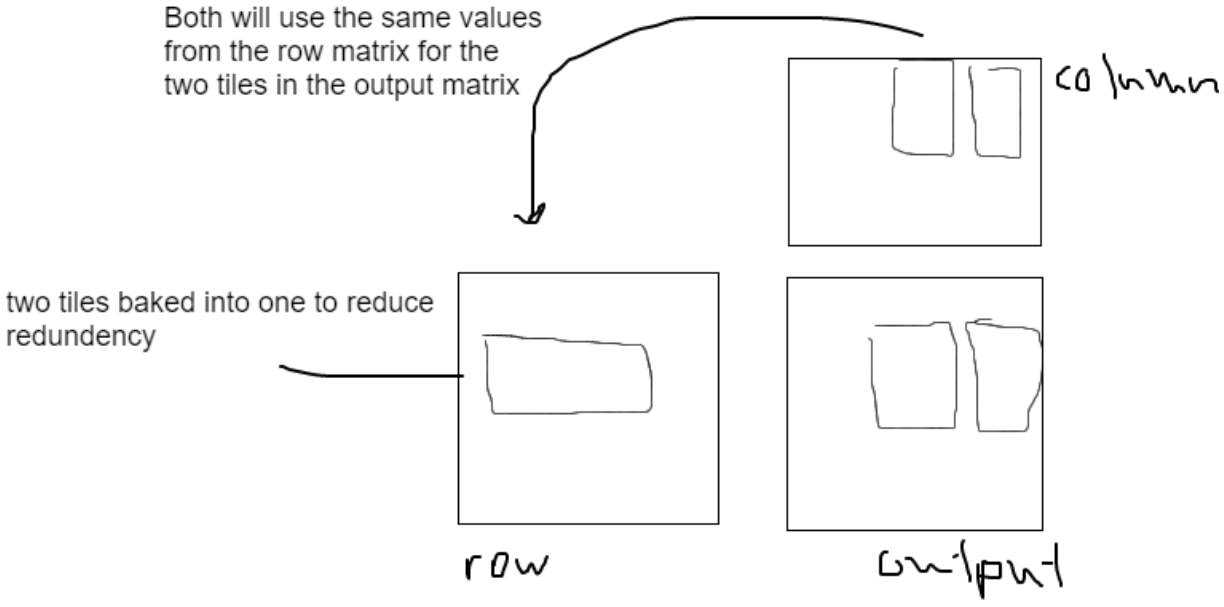
Rätt. 2 av 2 poäng.

12

Explain what thread granularity coarsening is. Give an example when thread granularity coarsening can make a program run faster. Also, explain why thread granularity coarsening could make our program run slower.

Thread coursing is the concept (if utilized correctly) of reducing redundancy in applications to avoid many global memory accesses. In the example of matrix multiplication with tiles, tiles in the same row (different tiles but same row) of the output matrix will use the same values from the row matrix many times over. So there is a lot of re-use of values. If the same value needs to be accessed from global memory in many different times, it is much better to increase the tile size per thread block thus allowing the values to be stored in shared memory once and reused from for the entire row (ideally). There is a good picture in the GPU book on how it can look like.

Thread coarsening is not as effective when redundancy is not a problem, then, you're loading more data per threadblock unnecessarily, because remember, a threadblock will take care of more operations and data now. This can make our program run slower because each thread block has a finite amount of shared memory and registers, using more of them will result in fewer threadblocks per SM which leads to performance cliff (significant reduction in parallisim because of slight tweaks to the parameters).



Ord: 203

Besvarad.

13

To make our program run faster we would like to overlap communication and computation. In the CUDA part of the course we discussed scenarios when such overlapping was possible. Describe one scenario when we can improve performance by overlapping computation and communication in a CUDA program.

CUDA streams allow for asyncronization between computation and data transfer for two different streams. An example of when this is useful is for 3d convolution where the halo values are required. This can either be done sequentially by each block calculating its own elements and then send the halo parts of that to adjacent blocks or done more efficiently by CUDA streams where a block calculates its halo blocks first then sends it to the adjacent blocks while they are doing their calculations on their internal values. The operation of transferring and computing data can be parallized by using streams, in this way, we can load the data on one stream and calculate on another

Ord: 115

Besvarad.

14 Privatization

Privatization is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel histogramming. Explain how privatization works and why privatization can improve performance.

Privitization works by having each threadblock or sometimes thread have its own private version of the global data. In essence, a private version of what exists on the main memory. This is done to alleviate all the conflicting writes that would occur from histogramming for example. Why? Because we don't want threads to conflict and overwrite each others values, so we add atomicity. Thus increasing the time per warp and leading to very long queues. We try to avoid atomicty as much as possible because it is bad for performance, therefore it is much better to have a private version of the array in shared memory so that only the threads in the threadblock can access it. And because it is in shared memory, the threads will have much faster access times thus increasing bandwidth and throughput, whilst decreasing conflicts between threads. The difficulty comes when the arrays in the shared memory need to be updated on global memory. In the example of histogram, each array can be combined to a single array and their values added correctly and then sent to global memory or host device. However in more complex situations, it requires more carefulness and algorithmic knowledge to combine all the values from all shared memories into the correct version.

Ord: 212

Besvarad.

15

For sparse matrix computation we discussed two optimizations: 1) sorting the rows so that the row with the largest number of non-zero elements comes first and the row with the smallest number of non-zero elements comes last, 2) transposing the matrix. Explain why and how each of these two optimizations can improve the performance of a CUDA program.

1) We use JDS to sort the matrix from longest row to shortest row to increase regularization. If we divide the sorted CSR matrix into sections, we will have sections that are close to each in row length. We do this to decrease thread divergence, thus increasing control flow and if we decide to add padding then the padded matrix won't be as large as it could have been thus decreasing the amount of space that could have been needed if we decided to pad the CSR matrix directly. By padding we can also allow for good warp effectivness and utilize memory bursting, which leads us into point 2.

2)
We transpose a CSR matrix because we want to utilize memory coalescing. In the CSR matrix, each thread takes care of a row usually, because that is how CSR matrix is created in a parallel implementation. This decreases the control flow of warps because the some rows are shorther than other rows, thus leading one thread to exit or wait before the other threads. When we transpose, the rows in memory will be sequentially layed out and therefore sequentially accessed by the warps thus decreasing thread divergence, and enables utlization of memory bursting within a warp.

Ord: 206

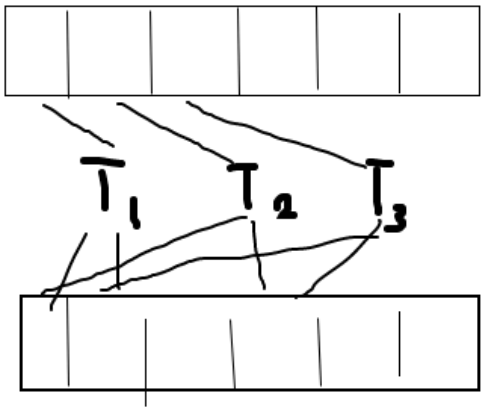
Besvarad.

16 Scatter to Gather

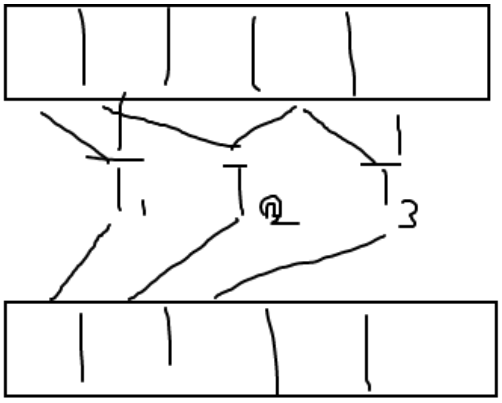
Scatter to gather conversion is a technique that can be used for improving the performance of some CUDA program. Explain how scatter to gather conversion works and why this technique can improve performance.

Scatter to gather is a technique used to convert a scatter algorithm where each thread accesses one position from the input array and through some function, writes to an output array to a gather opertation. A gather algorithm works opposite to how a scatter works, each threads accesses one position from the output array and through some inverted function, reads what values should be combined from the input array (see picture). The problem with scatter arises when multiple threads try to access the same output array position thus there will be conflicting writes between the threads, and to solve this, atomic operations are needed. We know that atomic operations reduce performance so they are to be avoided as much as possible. Therefore, a conversion from scatter to gather is benefical because now, each thread takes care of one output position so there will be no conflicting writes, there will however be conflicting reads from the input array but since we are only reading, it won't matter, thus eliminating the need for atomicity -> increase in performance.

(



Scatter: each can write to same but read from different parts



Gather: each write to one place but can read from the same

Ord: 177

Besvarad.

17 Tiling

Tiling is a technique that can be used for improving the performance of some CUDA programs, e.g., programs that perform parallel matrix multiplication or convolution. Explain how tiling works and why and how we may need to use __synchthreads() when using tiling.

Tiling is a data transformation technique that is used to improve spatial and temporal data locality by transferring "tiles" from global memory to shared memory. This allows the threads to work together by making the have almost the same access times to memory and decreasing congestion. Each thread block takes care of one tile (usually) and as long as the data they are working on is independent, the thread blocks can each work on their respective tile in parallel without any conflicts between them. However, before they start their work, the thread blocks need to load in their data from global memory to the shared memory, that is where syncing is required. Each thread loads their respective data into shared memory, once they have all loaded in their required data; computation can start, it is however very important to make all threads wait until all data that is necessary is loaded into shared memory to avoid any conflicts or null accesses.

Ord: 161

Besvarad.